

The μ OS++ IIIe test framework

**package.json, xpm, xPacks, CMake and the build matrix on
xpack-development**

Kernel: micro-os-plus-iii

Branch: xpack-development

How the μ OS++ IIIe test framework works

- 1. The mental model in one page
- 2. Directory map
- 3. xpm and xPacks
- 4. The package.json files
- 5. The configuration matrix
- 6. Where the dependencies land
- 7. The CMake include chain
- 8. The CMake target graph
- 9. The test sources
- 10. How a test is built and run
- 11. Common workflows
- 12. Continuous integration
- 13. Extending the framework
- 14. Generated files and what is ignored
- 15. Why it is built this way

Part II — Under the hood

- 16. From reset to os_main
- 17. Tracing and how test output reaches the host
- 18. Memory layout and linker scripts
- 19. The instrumented test and the board demos
- 20. Code style, formatting and editor integration
- 21. Legacy tests and history
- 22. Known limitations and caveats
- 23. The wider xPack ecosystem
- Appendix A — file-by-file quick reference
- Appendix B — glossary

How the μ OS++ IIIe test framework works

An analysis of the `xpack-development` branch of `micro-os-plus-iii`

This document explains, in plain English, how the test framework of the μ OS++ IIIe portable library actually works: what the `package.json` files contain, how **xpm** and **xPacks** fit in, how the **CMake** files are wired together, and how a test is finally built and run.

It describes the state of the `xpack-development` branch (the branch where all development happens, later merged into `xpack`). The reference commit used while writing is `c47f806b` ("ci.yml update deps"). Where the `smp` branch differs, this document is only about `xpack-development`.

1. The mental model in one page

The whole strategy is easy to say and harder to implement:

Write the tests once, then compile and run the same sources with as many toolchains as possible, on as many platforms as possible.

Everything in `tests/` exists to make that sentence true. The pieces are:

Piece	Role
Test sources	Platform-agnostic C/C++ programs in <code>tests/sources/<name></code> . Never modified per platform.
Platform glue	<code>tests/platforms/<name>/</code> — the linker script, the device, the CPU flags, and the command that launches the test.
CMake	Builds the images and registers each one with CTest.
CTest	Actually runs the tests and reports pass/fail.
xpm	The Node.js command-line tool that installs pinned tools, expands the build matrix, and runs named commands called actions .
xPacks	The packages (source libraries and binary tools) that xpm installs into <code>xpacks/</code> .
<code>package.json</code>	The manifest that declares the dependencies, the build configurations, and the actions.

A single command such as

```
xpm run test-all -C tests
```

walks the whole matrix: it installs nothing new, but for every declared configuration it configures CMake, builds the test images, and asks CTest to run them.

2. Directory map

The framework lives under `tests/` in the repository root:

```
micro-os-plus-iii/
├─ package.json          # the library itself (the xPack being tested)
├─ CMakeLists.txt        # builds the library target `micro-os-plus::iii`
├─ include/ src/         # the portable library sources
├─ tests/
│   ├─ package.json      # THE test manifest (deps, configs, actions)
│   ├─ package-lock.json # npm lockfile for the JS devDependencies
│   ├─ CMakeLists.txt    # CMake entry point for the tests
│   ├─ cmake/
│   │   ├─ global-definitions.cmake # which tests are enabled
│   │   ├─ common-options.cmake     # compile/link options common to all
│   │   └─ tests-main.cmake         # the orchestration script
│   ├─ sources/                  # the platform-agnostic test programs
│   │   ├─ rtos-apis/
│   │   ├─ mutex-stress/
│   │   ├─ cmsis-os-validator/
│   │   ├─ blinky/              # only on nucleo-f411re
│   │   └─ instrumentation/     # only on nucleo-f411re
│   ├─ platforms/              # the per-platform glue
│   │   ├─ native/
│   │   ├─ qemu-cortex-m0/
│   │   ├─ qemu-cortex-m3/
│   │   ├─ qemu-cortex-m4f/
│   │   ├─ qemu-cortex-m7f/
│   │   ├─ raspberrypi-pico/
│   │   ├─ nucleo-f411re/
│   │   ├─ nucleo-f767zi/
│   │   └─ nucleo-h743zi/
│   ├─ device-qemu-cortexm/    # generic QEMU device (vectors, startup, linker)
│   ├─ deprecated/            # old tests, kept for history, not built
│   ├─ xpacks/                # GENERATED by `xpm install` (git-ignored)
│   └─ build/                 # GENERATED build folders (git-ignored)
```

Two directories are generated and never committed:

- `tests/xpacks/` — the installed xPack dependencies (source and tools).

- `tests/build/<configuration>/` — one CMake binary directory per configuration, plus a nested `xpacks/` with configuration-specific dependencies.

Both are listed in `.gitignore`.

3. xpm and xPacks

xpm is a portable Node.js command-line application. It complements npm with features specific to C/C++ projects: it knows how to install source libraries and pre-built toolchains, and it reads a special `xpack` section in `package.json` to define build configurations and actions.

xPack is the package format. A source xPack is just a folder with a `package.json` (with an `xpack` section) and some sources. A binary xPack is a pre-built toolchain (compiler, QEMU, OpenOCD, CMake, Ninja, ...) with a `package.json` that describes where its binaries live.

Key concepts used throughout this document:

Term	Meaning
configuration	One combination of platform + toolchain + build type, e.g. <code>qemu-cortex-m7f-cmake-debug</code> . Each has its own build folder.
action	A named command in <code>package.json</code> , run as <code>xpm run <action> --config <configuration></code> .
devDependency	A package xpm installs into <code>tests/xpacks/</code> (top level) or into <code>build/<config>/xpacks/</code> (per configuration).
build-helper	The <code>@micro-os-plus/build-helper</code> xPack that provides the shared CMake functions and toolchain files.

The minimum xpm version is declared in `package.json`:

```
"xpack": {
  "minimumXpmRequired": "0.20.8"
}
```

4. The `package.json` files

There are two manifests that matter: the repository root one (the library being tested) and `tests/package.json` (the framework itself).

4.1 The root `package.json`

This describes the `@micro-os-plus/micro-os-plus-iii` library:

```
{
  "name": "@micro-os-plus/micro-os-plus-iii",
  "version": "7.1.0",
  "xpack": {
    "minimumXpmRequired": "0.20.8",
    "devDependencies": {
      "@xpack-dev-tools/clang": "21.1.8-1.1"
    },
    "actions": {
      "clang-format": "bash scripts/clang-format.sh",
      "cmake-format": "bash scripts/cmake-format.sh"
    }
  }
}
```

It contributes two things to the tests: the library source tree, and a pinned clang (which also supplies `clang-format`) used by the two code-style actions. The library package itself has no runtime dependencies: it is a monolithic source library, and the platform-specific parts are added by the applications (the tests) as separate xPacks. It is the package that consumers add with `xpm install github:micro-os-plus/micro-os-plus-iii`.

4.2 `tests/package.json`

This is the heart of the test framework. It has four important sections: `devDependencies`, `properties`, `actions`, and `buildConfigurations`.

devDependencies

```
"devDependencies": {
  "@xpack-dev-tools/cmake": "3.26.5-1.1",
  "@xpack-dev-tools/ninja-build": "1.11.1-3.1",
  "@micro-os-plus/build-helper": {
    "specifier": "^2.17.0",
    "local": "link",
    "platforms": "all"
  },
  "@xpacks/arm-cmsis-rtos-validator": {
    "specifier": "github:xpacks/arm-cmsis-rtos-validator#v1.0.0-4",
    "local": "link",
    "platforms": "all"
  },
  "@xpacks/chan-fatfs": {
    "specifier": "github:xpacks/chan-fatfs#v0.13.1-5",
    "local": "link",
    "platforms": "all"
  }
}
```

These are the packages needed by *every* configuration:

- **cmake** and **ninja-build** — the build generator and build tool.
- **build-helper** — supplies the `xpack_*` CMake functions and the toolchain files (`gcc.cmake`, `clang.cmake`, `arm-none-eabi-gcc.cmake`).
- **arm-cmsis-rtos-validator** — the Arm CMSIS RTOS API validator used by the `cmsis-os-validator` test.
- **chan-fatfs** — the FAT file system used by the `rtos-apis` test.

The `"local": "link"` setting means xpm creates a symbolic link to the copy in the central xPacks store instead of copying files; this makes development faster and keeps a single copy on disk.

properties

Properties are string templates used by the actions:

```
"properties": {
  "buildFolderRelativePath": "{{ 'build' | path_join: configuration.name | to_filename |
downcase }}",
  "commandCMakeReconfigure": "cmake -S . -B {{ properties.buildFolderRelativePathPosix }} -G Ninja
-D CMAKE_BUILD_TYPE={{ properties.buildType }} -D PLATFORM_NAME={{ properties.platformName }}",
  "commandCMakeBuild": "cmake --build {{ properties.buildFolderRelativePathPosix }}",
  "commandCMakePerformTests": "cd {{ properties.buildFolderRelativePath }} && ctest -V"
}
```

Notice that the build folder name is derived from the configuration name, and that the platform is passed to CMake as `-D PLATFORM_NAME=...`. That single variable is what makes the same `tests/CMakeLists.txt` behave differently for each platform.

actions

Actions are named commands. There are two families:

- **Per-configuration actions** — `prepare`, `build`, `test`, `clean`, `install`, `link-deps`. These are defined inside the hidden build configurations and are run with `--config`.
- **Aggregate actions** — `test-ci`, `test-all`, `test-native-cmake`, `test-cortex-cmake`, `install-ci`, `install-all`, `clean-all`, ... These call the per-configuration actions for many configurations in sequence.

Examples:

```
"test": [
  "xpm run test-qemu-cortex-m7f-cmake"
],
"test-ci": [
  "{% if os.platform == 'linux' %}xpm run test-native-cmake-gcc{% endif %}",

  "{% if os.platform == 'linux' or os.platform == 'darwin' %}xpm run test-native-cmake-clang{% endif
%}",
  "xpm run test-qemu-cortex-m0-cmake",
  "xpm run test-qemu-cortex-m3-cmake",
  "xpm run test-qemu-cortex-m7f-cmake"
],
"test-all": [
  "xpm run test-native-cmake",
  "xpm run test-cortex-cmake"
]
```

The `{% ... %}` parts are **Liquid** templates, evaluated by xpm with variables such as `os.platform`. This is how the same manifest works on Linux, macOS and Windows.

buildConfigurations

This is the matrix. Rather than repeating the platform, toolchain and build type for every combination, the manifest uses **inheritance**: small hidden building blocks are combined by name.

The hidden blocks are:

Block	What it adds
<code>cmake-actions</code>	<code>prepare</code> (configure with toolchain), <code>build</code> (reconfigure + build), <code>test</code> (<code>ctest -V</code>), <code>clean</code>
<code>native-actions</code>	<code>install</code> , <code>link-deps</code> for native
<code>cortexm-actions</code>	<code>install</code> , <code>link-deps</code> for Cortex-M
<code>native-dependencies</code>	<code>@micro-os-plus/posix-arch</code> , <code>@xpack-3rd-party/libucontext</code>
<code>cortexm-dependencies</code>	<code>@micro-os-plus/micro-os-plus-iii-cortexm</code>
<code>arm-cmsis-dependencies</code>	<code>@xpacks/arm-cmsis</code>
<code>arm-none-eabi-gcc-dependencies</code>	the <code>arm-none-eabi-gcc</code> 14.2.1 toolchain
<code>gccNN-dependencies</code>	pinned GCC 11/12/13/14 toolchains
<code>clangNN-dependencies</code>	pinned clang 13...19 toolchains
<code>qemu-arm-dependencies</code>	the QEMU ARM emulator
<code>openocd-dependencies</code>	OpenOCD for physical boards
<code>short-win-paths-properties</code>	short build-folder names on Windows (path-length limit)

A concrete configuration then looks like this:

```
"qemu-cortex-m7f-cmake-debug": {
  "inherit": [
    "cortexm-actions",
    "cmake-actions",
    "cortexm-dependencies",
    "arm-cmsis-dependencies",
    "arm-none-eabi-gcc-dependencies",
    "qemu-arm-dependencies",
    "short-win-paths-properties"
  ],
  "properties": {
    "buildType": "Debug",
    "platformName": "qemu-cortex-m7f",
    "toolchainFileName": "arm-none-eabi-gcc.cmake",
    "shortConfigurationName": "m7fd"
  }
}
```

The release variant inherits the debug one and only overrides `buildType`:

```
"qemu-cortex-m7f-cmake-release": {
  "inherit": ["qemu-cortex-m7f-cmake-debug"],
  "properties": {
    "buildType": "MinSizeRel",
    "shortConfigurationName": "m7fr"
  }
}
```

Note that release builds use `MinSizeRel` (`-Os`), not `Release`, on the embedded targets.

5. The configuration matrix

The complete set declared in `tests/package.json` is:

Platform	Configurations	Toolchain	Runner
native	native-cmake-sys , native-cmake-gcc , native-cmake-gcc11..14 , native-cmake-clang , native-cmake-clang13..19 (each debug + release)	host gcc/clang	run as a process
qemu-cortex-m0	qemu-cortex-m0-cmake-{debug, release}	arm-none-eabi-gcc	QEMU mps2-an385
qemu-cortex-m3	qemu-cortex-m3-cmake-{debug, release}	arm-none-eabi-gcc	QEMU mps2-an385
qemu-cortex-m4f	qemu-cortex-m4f-cmake-{debug, release}	arm-none-eabi-gcc	QEMU mps2-an386
qemu-cortex-m7f	qemu-cortex-m7f-cmake-{debug, release}	arm-none-eabi-gcc	QEMU mps2-an500
raspberrypi-pico	raspberrypi-pico-cmake-{debug, release}	arm-none-eabi-gcc	OpenOCD + RP2040
nucleo-f411re	nucleo-f411re-cmake-{debug, release}	arm-none-eabi-gcc	OpenOCD + ST-Link
nucleo-f767zi	nucleo-f767zi-cmake-{debug, release}	arm-none-eabi-gcc	OpenOCD + ST-Link
nucleo-h743zi	nucleo-h743zi-cmake-{debug, release}	arm-none-eabi-gcc	OpenOCD + ST-Link

The `native-cmake-sys` configuration is special: it does **not** use a toolchain file, so it uses whatever `gcc` / `g++` the system provides. The `native-cmake-gcc` and `native-cmake-clang` configurations download pinned xPack toolchains so that a version mismatch cannot hide a bug.

6. Where the dependencies land

xpm installs packages in two places, and this is important to understand when reading the CMake files:

- Top-level devDependencies** (from the main `devDependencies` block) go into `tests/xpacks/`:
`tests/xpacks/@micro-os-plus/build-helper` `tests/xpacks/@xpack-dev-tools/cmake` `tests/xpacks/@xpack-dev-tools/ninja-build` `tests/xpacks/@xpacks/arm-cmsis-rtos-validator` `tests/xpacks/@xpacks/chan-fatfs` This is why `tests/CMakeLists.txt` can do `list(APPEND CMAKE_MODULE_PATH ".../tests/xpacks/@micro-os-plus/build-helper/cmake")`.
- Per-configuration devDependencies** (declared by the inherited dependency blocks) go into `tests/build/<configuration>/xpacks/`:
`tests/build/qemu-cortex-m7f-cmake-debug/xpacks/@micro-os-plus/micro-os-plus-iii-cortexm` `tests/build/qemu-cortex-m7f-cmake-debug/xpacks/@xpacks/arm-cmsis` `tests/build/native-cmake-gcc-debug/xpacks/@micro-os-plus/micro-os-`

```
plus-iii-posix-arch      tests/build/native-cmake-gcc-debug/xpacks/@xpack-3rd-party/
libucontext             The platform dependencies-folders.cmake refers to these with $
{CMAKE_BINARY_DIR}/xpacks/...
```

The toolchain binaries (compilers, QEMU, OpenOCD) are also installed under `tests/build/<configuration>/xpacks/`, and xpm adds their `bin` folders to the `PATH` for the duration of the action.

7. The CMake include chain

CMake is invoked from `tests/`, so `tests/CMakeLists.txt` is the entry point. It runs a fixed sequence:

```
tests/CMakeLists.txt
├─ project(...)
├─ enable_language(C CXX ASM)
├─ set C11 / C++20 standards
├─ enable_testing()                # turn on CTest
├─ append build-helper/cmake to CMAKE_MODULE_PATH
├─ include("micro-os-plus-build-helper") # the xpack_* functions
├─ xpack_display_greetings(...)
├─ include("cmake/global-definitions.cmake") # ENABLE_*_TEST switches
└─ include("cmake/tests-main.cmake")       # the real orchestration
```

7.1 tests-main.cmake

This is the generated orchestration file. In order it:

1. Enables `compile_commands.json` for IDEs.
2. Sets `.elf` as the executable suffix when cross-compiling (`CMAKE_SYSTEM_NAME` `STREQUAL` `"Generic"`).
3. Includes `cmake/common-options.cmake`, which defines the interface target `micro-os-plus::common-options` with the common warnings and flags.
4. Includes `platforms/${PLATFORM_NAME}/cmake/definitions.cmake`, which sets `xpack_platform_compile_definition`, `xpack_device_compile_definition`, and the linker script name.
5. Includes `platforms/${PLATFORM_NAME}/cmake/dependencies-folders.cmake`, which lists the folders to be added as subdirectories.
6. Calls `xpack_add_dependencies_subdirectories(...)`, which runs `add_subdirectory()` for each dependency folder listed in step 5. This is how `sources/rtos-apis`, the Cortex-M port, the CMSIS device and the filesystem all become part of the same build graph.
7. Calls `add_subdirectory("../top-bin")` — this adds the repository root `CMakeLists.txt`, defining the library target `micro-os-plus::iii`.

8. Calls `add_subdirectory("platforms/${PLATFORM_NAME}" "platform-bin")`, which defines the platform library and registers the test executables.

The key line is step 8: **the platform name alone selects everything else**. `PLATFORM_NAME` arrives from the xpm action as `-D PLATFORM_NAME=...`.

7.2 `common-options.cmake`

Creates an interface library, `micro-os-plus-common-options-interface`, and its alias `micro-os-plus::common-options`. It carries:

- compile definitions `DEBUG` and `TRACE` in Debug builds,
- `OS_USE_OS_APP_CONFIG_H` (tells the library to include the application config header),
- `-fmessage-length=0 -fsigned-char -ffunction-sections -fdata-sections`,
- the full warning set from `xpack_set_all_compiler_warnings()`.

Every test executable links this interface library, so all sources — including the tested library — are compiled with the same options and warnings.

7.3 `global-definitions.cmake`

Simple on/off switches for the portable tests:

```
set (ENABLE_RTOS_APIS_TEST true)
set (ENABLE_MUTEX_STRESS_TEST true)
set (ENABLE_CMSIS_OS_VALIDATOR_TEST true)
```

Individual platforms add their own (for example `nucleo-f411re` sets `ENABLE_BLINKY_TEST` and `ENABLE_INSTRUMENTATION_TEST` in its `definitions.cmake`).

7.4 The `xpack_*` helper functions

These come from `@micro-os-plus/build-helper/cmake/micro-os-plus-build-helper.cmake`:

Function	Purpose
<code>xpack_add_dependencies_subdirectories(list binpath)</code>	<code>add_subdirectory()</code> each folder, failing if a <code>CMakeLists.txt</code> is missing.
<code>xpack_set_all_compiler_warnings(var)</code>	Builds a long list of warnings (<code>-Wall</code> , <code>-Wextra</code> , <code>-Wconversion</code> , <code>-Wshadow</code> , ...) tuned per compiler and version.
<code>xpack_display_target_lists(target)</code>	Verbose dump of a target's includes, sources, definitions and options.
<code>xpack_display_greetings(package.json)</code>	Verbose banner with CMake/compiler/package versions.
<code>xpack_add_cross_custom_commands(target)</code>	POST_BUILD step that runs <code>size</code> on the ELF, and optionally <code>objcopy</code> / <code>objdump</code> .
<code>xpack_get_package_name_and_version(path)</code>	Reads <code>name</code> and <code>version</code> out of a <code>package.json</code> with CMake's JSON support.

8. The CMake target graph

The build is entirely built from **INTERFACE libraries** — targets with no compiled objects of their own that propagate sources, include folders, definitions and options to whatever links them. This is what allows the same source files to be compiled once per executable with the right flags.

The targets are:

Target (alias)	Defined in	Carries
<code>micro-os-plus::iii</code>	root <code>CMakeLists.txt</code>	the whole portable library: <code>include/</code> and every file in <code>src/</code>
<code>micro-os-plus::common-options</code>	<code>tests/cmake/common-options.cmake</code>	warnings and common flags
<code>micro-os-plus::platform</code>	<code>platforms/<name>/cmake/platform-library.cmake</code>	platform include folder, CPU flags, linker script, <code>_POSIX_C_SOURCE</code> , and links to the port + device
<code>micro-os-plus::device</code>	<code>tests/device-qemu-cortexm/CMakeLists.txt</code>	startup, vectors, exception handlers and linker scripts for QEMU
<code>test::rtos-apis</code>	<code>tests/sources/rtos-apis/CMakeLists.txt</code>	the RTOS API test sources
<code>test::mutex-stress</code>	<code>tests/sources/mutex-stress/CMakeLists.txt</code>	the mutex stress test sources
<code>test::cmsis-os-validator</code>	<code>tests/sources/cmsis-os-validator/CMakeLists.txt</code>	(only the config header; the validator provides the code)
<code>test::blink</code> , <code>test::instrumentation</code>	their <code>sources/</code> folders	platform-specific demos
<code>xpacks::chan-fatfs</code>	installed xPack	FAT file system
<code>xpacks::arm-cmsis-os-validator</code>	installed xPack	Arm CMSIS RTOS validator
<code>micro-os-plus::iii-cortexm</code>	installed xPack	Cortex-M scheduler port
<code>micro-os-plus::iii-posix-arch</code>	installed xPack	synthetic POSIX scheduler port
<code>xpack-3rd-party::libucontext</code>	installed xPack	ucontext support for the POSIX port
<code>xpacks::arm-cmsis</code>	installed xPack	CMSIS core headers

Each test executable links a fixed set: the common options, the test library, the tested library `micro-os-plus::iii`, any portable dependency, and the platform library. For example the QEMU `rtos-apis-test` links:

```
target_link_libraries (rtos-apis-test PRIVATE
    micro-os-plus::common-options
    test::rtos-apis
    micro-os-plus::iii
    xpacks::chan-fatfs
    micro-os-plus::platform
)
```

Because the platform library links `micro-os-plus::iii-cortexm` and `micro-os-plus::device`, and the device links `xpacks::arm-cmsis`, the linker command line ends up containing everything needed, with no manual ordering.

9. The test sources

The portable tests are plain programs with an `os_main()` entry point, not `main()`. The library provides a weak default `main()` in `src/rtos/os-main.cpp`: it prints the banner, starts the scheduler, and runs `os_main()` on a dedicated μ OS++ thread. That is why every test only has to define `os_main()` and can use the RTOS API directly. The same test source is compiled once per platform; only the platform glue changes.

rtos-apis

The broadest test. `src/main.cpp` calls a series of test functions, each in its own file:

- `test-cpp-api.cpp` — the C++ RTOS API,
- `test-c-api.c` — the C API,
- `test-iso-api.cpp` — the ISO C++ API (`std::thread`, `std::mutex`, ...),
- `test-posix-io-api.cpp` — the POSIX I/O API,
- `test-chan-fatfs.cpp` — the FAT file system,
- `test-cmsis-os1.cpp` — the CMSIS-OS v1 compatibility API.

Each returns 0 on success. `main.cpp` stops at the first failure and returns the error code, which becomes the process exit code and thus the CTest result.

mutex-stress

Several threads contend for a mutex with random lock durations; the test verifies the scheduling distribution is fair. It is designed to expose race conditions that a simple functional test would miss.

cmsis-os-validator

Wraps the Arm CMSIS RTOS API validator. The test folder contains only an `os-app-config.h`; the actual validator code and `main` come from the `@xpacks/arm-cmsis-rtos-validator` xPack.

blinky and instrumentation

Platform-specific demos, enabled only where they make sense (the `nucleo-f411re` board). `blinky` toggles the LED; `instrumentation` exercises SEGGER SystemView tracing.

Every test folder provides its own `include/cmsis-plus/os-app-config.h`. This is the application configuration header: it defines the system tick frequency, stack sizes, memory pool sizes, which traces are enabled, and so on. The `OS_USE_OS_APP_CONFIG_H` definition added by `common-options.cmake` is what makes the library include it.

10. How a test is built and run

10.1 The native platform

`tests/platforms/native/CMakeLists.txt` creates a normal host executable and registers it directly:

```
add_test (NAME "rtos-apis-test" COMMAND rtos-apis-test)
```

CTest runs the binary as a process and uses its exit code. On Linux and macOS the platform library also computes the compiler's runtime library paths and adds `-Wl, -rpath, ...` so the executable can find `libstdc++` / `libc++`.

10.2 The QEMU platforms

`tests/platforms/qemu-cortex-m7f/CMakeLists.txt` creates an ELF and registers CTest to launch QEMU with semihosting:

```
add_test (NAME "rtos-apis-test"
  COMMAND qemu-system-arm --machine mps2-an500 --cpu cortex-m7
    --kernel rtos-apis-test.elf --nographic
    -d unimp,guest_errors
    --semihosting-config enable=on,target=native,arg=rtos-apis-test)
```

`--semihosting-config enable=on,target=native` makes the guest's semihosting calls (standard output and exit) go to the host. The QEMU process therefore prints the test output and exits with the guest's exit code, which is exactly what CTest needs. The mapping is:

Platform	Machine	CPU	Linker script
<code>qemu-cortex-m0</code>	<code>mps2-an385</code>	<code>cortex-m3</code>	<code>mem-mps2-an385.ld</code>
<code>qemu-cortex-m3</code>	<code>mps2-an385</code>	<code>cortex-m3</code>	<code>mem-mps2-an385.ld</code>
<code>qemu-cortex-m4f</code>	<code>mps2-an386</code>	<code>cortex-m4</code>	<code>mem-mps2-an386.ld</code>
<code>qemu-cortex-m7f</code>	<code>mps2-an500</code>	<code>cortex-m7</code>	<code>mem-mps2-an500.ld</code>

The M0 configuration is run on the `mps2-an385` machine with the Cortex-M3 CPU because QEMU has no separate M0 board; the code is still compiled for `cortex-m0`.

10.3 The physical boards

For the NUCLEO boards and the Raspberry Pi Pico, CTest launches **OpenOCD**, which programs the flash and starts execution:

```

add_test (NAME "rtos-apis-test"
  COMMAND openocd -c "gdb port disabled" -c "tcl port disabled"
    -f interface/stlink-dap.cfg -f target/stm32f4x.cfg
    -c "program rtos-apis-test.elf verify"
    -c "arm semihosting enable"
    -c "arm semihosting_cmdline rtos-apis-test"
    -c "reset")

```

Semihosting again carries the output and the exit code back to the host over the debug probe. The Pico uses a different OpenOCD configuration and a `rp2040.core0 arp_reset assert 0` sequence to release the core after reset.

10.4 The build sequence

For any configuration, the actions do the following:

1. `prepare` — `cmake -S . -B build/<config> -G Ninja -D CMAKE_BUILD_TYPE=... -D PLATFORM_NAME=... --log-level=VERBOSE` plus `-D CMAKE_TOOLCHAIN_FILE=xpacks/@micro-os-plus/build-helper/cmake/toolchains/<file>`.
2. `build` — re-run the configure step, then `cmake --build build/<config>`.
3. `test` — `cd build/<config> && ctest -V`.
4. `clean` — `cmake --build build/<config> --target clean`.

The toolchain files set the compilers and, for cross builds, the `CMAKE_SYSTEM_NAME Generic` / `CMAKE_SYSTEM_PROCESSOR arm` pair that marks the build as cross-compilation.

11. Common workflows

All commands are run from `tests/` (or with `-C tests` from the root).

```
# one-time: install the pinned tools and packages
xpm install
xpm install --config qemu-cortex-m7f-cmake-debug

# the default quick test (QEMU Cortex-M7, debug + release)
xpm run test

# everything CI runs on this OS
xpm run test-ci

# the full local matrix (native + all Cortex-M)
xpm run test-all

# one platform only
xpm run test-qemu-cortex-m3-cmake
xpm run test-native-cmake-clang19

# clean everything
xpm run clean-all
```

A typical single-configuration run expands to:

```
xpm run prepare --config qemu-cortex-m7f-cmake-debug
xpm run build   --config qemu-cortex-m7f-cmake-debug
xpm run test    --config qemu-cortex-m7f-cmake-debug
```

The `test-*` aggregate actions are just lists of those three commands for the debug and release variants of a platform.

12. Continuous integration

The GitHub Actions workflow is `.github/workflows/ci.yml`. It runs on a matrix of operating systems:

```
os: [ 'ubuntu-24.04', 'ubuntu-24.04-arm', 'macos-15-intel', 'macos-15', 'windows-2025' ]
```

and does four things:

1. Checks out the repository.
2. Installs Node.js 22 and `xpm@0.20.8`.
3. `npm --prefix tests ci` — installs the JavaScript devDependencies (`del-cli`) from `tests/package-lock.json`.

4. `xpm run install-ci -C tests` — installs the xPack dependencies for the configurations relevant to that OS.
5. `xpm run test-ci -C tests` — runs the tests.

`install-ci` and `test-ci` are Liquid-templated, so each operating system installs and runs only what it can:

- Linux runs native GCC and clang, plus QEMU M0/M3/M7.
- macOS runs native clang, plus QEMU M0/M3/M7.
- Windows runs the QEMU Cortex-M targets.

The README also mentions a second, manually triggered workflow (`test-all.yml`) that is intended to run every test on every supported platform; in the current branch that file is not present, and `xpm run test-all` is the local equivalent.

The CI workflow ignores documentation-only changes (`**/*.md`, `docs/**`, `doxygen/**`, ...) through `paths-ignore`, so a docs edit does not trigger a build. It also uses `concurrency` with `cancel-in-progress: true`, so pushing a new commit cancels the previous run for the same branch.

13. Extending the framework

Add a portable test

1. Create `tests/sources/<name>/` with a `CMakeLists.txt` that defines an interface library and an alias `test::<name>`, following the pattern of `sources/mutex-stress/`.
2. Add the sources and its `include/cmsis-plus/os-app-config.h`.
3. Add an `ENABLE_<NAME>_TEST` switch in `tests/cmake/global-definitions.cmake` (or a platform `definitions.cmake`).
4. Add the folder to each platform's `dependencies-folders.cmake`.
5. Register the executable and `add_test()` in each platform's `CMakeLists.txt`.

Add a platform

1. Create `tests/platforms/<name>/` with:
2. `CMakeLists.txt` (the test executables and `add_test()` commands),
3. `cmake/definitions.cmake` (device and platform compile definitions, linker script name, platform-specific test switches),
4. `cmake/dependencies-folders.cmake` (the folders to add),
5. `cmake/platform-library.cmake` (the `micro-os-plus::platform` interface library),
6. `include/cmsis-plus/platform.h`.
7. Add a `buildConfigurations` entry (or a family of entries) in `tests/package.json`, inheriting the appropriate hidden blocks and setting `platformName` and `toolchainFileName`.

8. Add aggregate `test-<name>-cmake` actions if desired.

Add a toolchain version

Add a hidden `gccNN-dependencies` / `clangNN-dependencies` block pinning the xPack tool, then a configuration that inherits it. No CMake changes are needed because the toolchain file name is already a property.

14. Generated files and what is ignored

Path	Generated by	Committed?
<code>tests/xpacks/</code>	<code>xpm install</code> (top-level devDependencies)	no
<code>tests/build/<config>/</code>	CMake + <code>xpm install --config</code>	no
<code>tests/build/<config>/xpacks/</code>	<code>xpm install --config</code>	no
<code>tests/build/<config>/compile_commands.json</code>	CMake	no
<code>tests/build/<config>/platform-bin/*.elf</code>	the build	no
<code>tests/build/<config>/platform-bin/*-map.txt</code>	the linker	no
<code>tests/node_modules/</code>	<code>npm install</code>	no
<code>tests/package-lock.json</code>	npm	yes

The `.gitignore` at the repository root ignores `build/`, `node_modules/` and `xpacks/`. The `tests/package.json` `deep-clean` action removes `build`, `node_modules`, `xpacks` and `package-lock.json` when a full reset is needed.

15. Why it is built this way

A few design decisions explain most of the structure:

- **Interface libraries everywhere.** Because every component is an INTERFACE library, the *same* source file can be compiled into many executables with different flags, without duplicating files or maintaining lists of objects.
- **One `PLATFORM_NAME` switch.** The platform name selects the definitions, the dependencies and the executables. There is no `if(OS)` branching in the top-level scripts.

- **Pinned tools as xPacks.** Every compiler, QEMU and OpenOCD comes from an xPack devDependency, so a developer's machine and CI use identical versions, and nothing has to be installed globally.
- **The matrix lives in `package.json`.** Adding a platform or a toolchain is a JSON edit, not a change to the build logic.
- **CTest is the common interface.** Whether the test runs as a native process, in QEMU, or on a physical board via OpenOCD, the result is still a CTest pass/fail with the program's output, so the aggregate actions are just lists of `ctest -V` calls.

Part II — Under the hood

Sections 1–15 describe how the framework is organised. This part follows what actually happens when a test runs: how the firmware starts, how its output reaches the host, how memory is laid out, and the supporting pieces around the build — instrumentation, formatting, editor integration, the legacy material, the known caveats, and the wider ecosystem the tests depend on.

16. From reset to `os_main`

A test defines `os_main()`, not `main()`. The entire boot path that reaches it is supplied by the library and the device package.

The sequence on a Cortex-M target is:

1. **Reset.** The CPU loads the initial stack pointer and `Reset_Handler` from the first two entries of `_interrupt_vectors[]`, the table defined in `tests/device-qemu-cortexm/src/vectors-cortexm.c` (or the equivalent `vectors_*.c` in a board's device folder). The table lives in the `.isr_vector` section, which the linker places at the start of flash.
2. `Reset_Handler` → `_start`. `Reset_Handler`, in `src/startup/exception-handlers.c`, calls `_start`, implemented in `src/startup/startup.cpp`.
3. `_start` performs, in order:
4. `os_startup_initialize_hardware_early()` — a weak hook implemented in `src/startup/initialize-hardware.c`. It calls the CMSIS `SystemInit()` (and `SystemClock_Config()` when `OS_HAS_SYSTEMCLOCK_CONFIG` is defined), then sets `SCB->VTOR` to the real vector table address.
5. copy `.data` from flash to RAM and clear `.bss` (or use the multi-region arrays when `OS_INCLUDE_STARTUP_INIT_MULTIPLE_RAM_SECTIONS` is defined).
6. `os_startup_initialize_hardware()` — a second weak hook; on QEMU it only calls `SystemCoreClockUpdate()`, on boards it initialises the clocks.
7. `os_startup_initialize_free_store()` — sets up the heap.
8. `os_startup_initialize_args()` — builds `argc` / `argv`; on embedded targets it can take them from semihosting.

9. run the preinit and init arrays, i.e. the C++ static constructors.
10. call `main(argc, argv)`.
11. `main()`. This is the library's weak default in `src/rtos/os-main.cpp`. It prints the μOS++ banner, creates a μOS++ thread with the stack size from `OS_INTEGER_RTOS_MAIN_STACK_SIZE_BYTES`, and starts the scheduler. The thread's trampoline prints the arguments, calls `os_main(argc, argv)`, and then calls `std::exit(code)`.
12. **Exit.** `std::exit()` runs the `atexit()` handlers and static destructors, then calls `_exit()`, which on an embedded target issues a semihosting `SYS_EXIT` with the code.

Because the reset vector, the startup code and `main()` all come from the library, a test file only has to provide `os_main()`; nothing platform-specific has to be written.

The **device package** is the part that differs per board:

Platform	Device sources
<code>qemu-cortex-m0/m3/m4f/m7f</code>	<code>tests/device-qemu-cortexm/</code> (generic QEMU device)
<code>nucleo-f411re</code>	<code>tests/platforms/nucleo-f411re/device/</code> + <code>stm32f411cubemx/</code>
<code>nucleo-f767zi</code>	<code>tests/platforms/nucleo-f767zi/device/</code>
<code>nucleo-h743zi</code>	<code>tests/platforms/nucleo-h743zi/device/</code>
<code>raspberrypi-pico</code>	<code>tests/platforms/raspberrypi-pico/device/</code> + <code>src/bs2_default_padded_checksummed.S</code>

The QEMU device is deliberately minimal: a 16 MHz `SystemCoreClock`, an FPU enable in `SystemInit()`, a generic vector table, and a SysTick handler that calls `os_systick_handler()`. The board devices add the CubeMX-generated `system_stm32*.c`, the real vector tables, and the STM32 HAL sources. All of them become the `micro-os-plus::device` interface library.

17. Tracing and how test output reaches the host

Two mechanisms carry text out of the target: the μOS++ **trace** facility and the **newlib retargeting** of `printf`.

The trace facility

The core is `src/diag/trace.cpp`, compiled only when the `TRACE` macro is defined — which `common-options.cmake` does for Debug builds. It provides `os::trace::printf()`, `write()` and `flush()`, plus the C alias `trace_printf()`.

The actual backend is chosen at compile time:

Macro	Backend file	Used by
<code>OS_USE_TRACE_SEMIHOSTING_STDOUT</code>	<code>src/diag/trace-semihosting.cpp</code>	QEMU and board tests
<code>OS_USE_TRACE_SEGGER_RTT</code>	<code>src/diag/trace-segger-rtt.cpp</code>	SEGGER RTT (instrumentation)
<code>OS_USE_TRACE_ITM</code>	<code>src/diag/trace-itm.cpp</code>	SWO/ITM debug output
(none)	<code>src/diag/trace.cpp</code> weak fallback	disabled output

Each platform's `CMakeLists.txt` sets the macro on the test executable. For example, the QEMU platform adds:

```
target_compile_definitions (${name} PRIVATE OS_USE_TRACE_SEMIHOSTING_STDOUT)
```

while the native platform adds `OS_USE_TRACE_POSIX_STDOUT`, which writes through the POSIX I/O layer to the host's standard output.

Newlib retargeting

The library also provides the C library system calls, so ordinary `printf()` works:

- On embedded targets, `src/semihosting/c-syscalls-semihosting.cpp` implements `_write`, `_read`, `_exit`, `_sbrk` and friends with semihosting calls.
- On native targets, `src/libc/_sbrk.c`, `src/libc/stdlib/*` and `src/posix-io/c-syscalls-posix.cpp` provide the POSIX equivalents.

How the result gets back

The chain is short and uniform:

```
os_main() returns int
└─ main trampoline calls std::exit(code)
    └─ _exit(code)
        └─ semihosting SYS_EXIT          (embedded)
            or process exit              (native)
                └─ QEMU / OpenOCD exits with that code
                    └─ CTest records pass or fail
```

The `--semihosting-config ...arg=<name>` value becomes `argv[0]`, which the trampoline prints with `trace::dump_args()`, so the CTest log shows which test ran. Nothing else is needed for CTest to decide the outcome.

18. Memory layout and linker scripts

Each device supplies two linker scripts, both passed with `-T` through `micro-os-plus::device`:

- `mem-<board>.ld` — the `MEMORY` block (flash and RAM origins and lengths).
- `sections-flash.ld` — the `SECTIONS` block: where the vector table, the init/fini arrays, `.text`, `.data`, `.bss`, `.noinit` and the stack check live.

The link uses `-nostartfiles` (so the library's own `_start` is used instead of the toolchain startup) and `-WL,--gc-sections` (so unused code is dropped). From GCC 12 the platform library also adds `-WL,--no-warn-rwx-segment`.

The QEMU memory maps are:

Platform	FLASH	RAM	PSRAM (heap)
<code>qemu-cortex-m0</code>	0x00000000, 8 MB	0x20000000, 8 MB	0x21000000, 16 MB
<code>qemu-cortex-m3</code>	0x00000000, 8 MB	0x20000000, 8 MB	0x21000000, 16 MB
<code>qemu-cortex-m4f</code>	0x00000000, 8 MB	0x20000000, 8 MB	0x60000000, 16 MB
<code>qemu-cortex-m7f</code>	0x00000000, 8 MB	0x20000000, 8 MB	0x60000000, 16 MB

The PSRAM region exists because the QEMU startup asks semihosting for `SYS_HEAPINFO` and places the heap and stack at the addresses QEMU reports; the `PSRAM` region simply reserves that space in the map. The physical boards use their own memory maps — for instance the Raspberry Pi Pico has 2 MB of flash at 0x10000000 and 256 KB of RAM at 0x20000000.

After linking, `xpack_add_cross_custom_commands()` always runs `size` on the ELF, and can optionally run `objcopy` (hex) and `objdump` (listing) when the `xpack_create_hex` / `xpack_create_listing` variables are set.

19. The instrumented test and the board demos

Two test sources are board-specific and are only built where they make sense.

instrumentation

Enabled on `nucleo-f411re` through `ENABLE_INSTRUMENTATION_TEST` in that platform's `definitions.cmake`. It exercises SEGGER SystemView. The platform builds two images:

- `instrumentation-test` — a simple case,
- `rtos-apis-instrumentation-test` — the full RTOS API test with instrumentation enabled.

Both `define` `OS_HAS_SYSTEMCLOCK_CONFIG`, `OS_USE_TRACE_SEMIHOSTING_STDOUT` and `OS_INCLUDE_INSTRUMENTATION`, and link the `segger::rtt` and `segger::system-view` targets, which come from the `@xpack-segger/rtt` and `@xpack-segger/system-view` devDependencies.

There is **no automated CTest entry** for these images: SystemView requires the SEGGER desktop tools and a J-Link. The platform folder ships the project files used for manual sessions (`instrumentation.jdebug`, `instrumentation.SVPrj`, `rtos-api-test-instrumentation.jdebug`).

blinky

Also `nucleo-f411re`-only, enabled by `ENABLE_BLINKY_TEST`. It toggles the board LED; the OpenOCD test passes two arguments (the on and off times in milliseconds) through `arm semihosting_cmdline`.

20. Code style, formatting and editor integration

The project enforces a consistent style with three tools, all configured from the `config/` folder:

- **clang-format** — `config/.clang-format` (and the root `.clang-format`) uses `BasedOnStyle: GNU` with C++20. The action `clang-format` runs `scripts/clang-format.sh`, which formats `src/`, `include/`, `tests/sources/` and `tests/platforms/`.
- **cmake-format** — `config/.cmake-format.py` sets a 2-space indent, an 80-column width and `dangle_parens`. The action `cmake-format` runs `scripts/cmake-format.sh` over every `CMakeLists.txt` and `*.cmake`, skipping `build/`, `xpacks/` and `node_modules/`.
- **prettier** — `config/.prettierrc.json` and `config/.prettierignore` cover the JavaScript/JSON files (for example the `package.json` files).

Both formatting actions are declared in the root `package.json`, so they are run from the repository root:

```
xpm run clang-format
xpm run cmake-format
```

For editing, the repository ships VS Code support. `.vscode/launch.json` contains `cppdbg` launch configurations for the native test executables, with the xPack tool folders added to `PATH` and `LD_LIBRARY_PATH` so the debugger finds the right runtime. `.vscode/settings.json` excludes the generated and third-party trees from indexing (`xpack.exclude`, `npm.exclude`) and adds the project's spelling dictionary. With the xPack VS Code extension installed, the xpm actions appear as buttons, so the same `prepare`/`build`/`test` steps can be run without typing commands.

21. Legacy tests and history

`tests/deprecated/` preserves the original tests that were used with the Eclipse Embedded CDT projects, before the current xpm/CMake framework existed. It is **not built** by any configuration. It contains:

- `gcc/` — early C and C++ serial/USB tests and a tiny syscalls file,
- `posix-drivers/` — circular buffer and USART tests,
- `posix-io/` — device, directory, file, pool and socket tests,
- `sema-stress/` — an early semaphore stress test,
- `scripts/` — the old shell scripts (`test-all.sh` , `trigger-workflow-test-all.sh` , ...),
- `README.md` — the historical instructions (clone the sibling repositories, symlink them into the xPacks store, import into Eclipse).

Some of these tests have not yet been ported to the current framework; they are kept so nothing is lost, and a few may be revived later.

22. Known limitations and caveats

The maintainers' notes in `tests/T0-CHECK.md` and `tests/README.md` record a few things worth knowing:

- **No trace output in the scheduler interrupt handler.** If trace prints happen inside the scheduler ISR, the tests occasionally fail — less often under emulation, almost always on physical boards. This is why the `rtos-apis` `os-app-config.h` leaves `OS_TRACE_RTOS_SCHEDULER` commented out. With that discipline the tests have run for many hours in loops without problems.
 - **Historical release-build failures.** Some macOS release runs of `rtos-apis-test` were recorded as `Illegal instruction` or `SegFault`; these were noted but not yet investigated, and debug builds pass.
 - **The manual `test-all.yml` workflow is not in the branch.** The README refers to it, but only `ci.yml` exists; `xpm run test-all` is the local equivalent.
 - **No Cortex-M0 machine in QEMU.** The `qemu-cortex-m0` configuration compiles for `cortex-m0` but runs on the `mps2-an385` (Cortex-M3) machine.
 - **Physical-board tests are manual.** They require the board and a debug probe (ST-Link or CMSIS-DAP), so they are not part of CI.
-

23. The wider xPack ecosystem

The tests do not exercise a self-contained library. The portable `micro-os-plus-iii` is combined with a port, a device and third-party xPacks, all pinned in `tests/package.json`:

xPack	Version	Role
@micro-os-plus/micro-os-plus-iii-cortexm	v1.1.0	Cortex-M scheduler port
@micro-os-plus/posix-arch	v1.0.1	synthetic POSIX scheduler port
@xpack-3rd-party/libucontext	v1.2.0-5	ucontext support for the POSIX port
@xpacks/arm-cmsis	v4.5.0-7	CMSIS core and device headers
@xpacks/chan-fatfs	v0.13.1-5	FAT file system used by rtos-apis
@xpacks/arm-cmsis-rtos-validator	v1.0.0-4	CMSIS RTOS API validator
@micro-os-plus/build-helper	^2.17.0	CMake functions and toolchain files

The **build-helper** is the shared infrastructure package. Besides the `xpack_*` functions used by the CMake scripts, it ships toolchain files for several architectures:

```
cmake/toolchains/gcc.cmake
cmake/toolchains/clang.cmake
cmake/toolchains/arm-none-eabi-gcc.cmake
cmake/toolchains/aarch64-none-elf-gcc.cmake
cmake/toolchains/riscv-none-elf-gcc.cmake
cmake/toolchains/riscv-none-embed-gcc.cmake
```

Only `gcc.cmake`, `clang.cmake` and `arm-none-eabi-gcc.cmake` are used on this branch; the AArch64 and RISC-V files are ready for future platforms. It also provides developer scripts (`dev-scripts/get-libraries-paths.sh`, used for the native RPATH, and the `git-*` helpers).

On Linux, `"local": "link"` makes xpm link each package to its copy in the central store (`~/.local/xPacks/...`) instead of copying files. The `tests/package.json` actions `git-clone-deps`, `git-pull-deps` and `link-deps` exist so a developer can clone the dependency repositories under `$HOME/work` and work on them in place, then have the tests use those working copies.

Appendix A — file-by-file quick reference

File	Read it for
<code>tests/package.json</code>	dependencies, build configurations, actions
<code>tests/CMakeLists.txt</code>	CMake entry point, <code>enable_testing()</code>
<code>tests/cmake/tests-main.cmake</code>	the include order and orchestration
<code>tests/cmake/common-options.cmake</code>	warnings and common flags
<code>tests/cmake/global-definitions.cmake</code>	which portable tests are on
<code>tests/platforms/<p>/cmake/definitions.cmake</code>	device/platform macros, linker script
<code>tests/platforms/<p>/cmake/dependencies-folders.cmake</code>	which folders become subdirectories
<code>tests/platforms/<p>/cmake/platform-library.cmake</code>	CPU flags, linker options, platform target
<code>tests/platforms/<p>/CMakeLists.txt</code>	test executables and <code>add_test()</code>
<code>tests/sources/<t>/CMakeLists.txt</code>	the <code>test::<t></code> interface library
<code>tests/device-qemu-cortexm/CMakeLists.txt</code>	the QEMU device target
<code>tests/device-qemu-cortexm/src/*</code>	vectors, startup hooks, exception handlers
<code>tests/device-qemu-cortexm/linker-scripts/*</code>	<code>mem-*.ld</code> memory map and <code>sections-flash.ld</code>
root <code>CMakeLists.txt</code>	the <code>micro-os-plus::iii</code> library target
<code>src/startup/startup.cpp</code>	<code>_start</code> , the reset-to- <code>main</code> boot path
<code>src/startup/initialize-hardware.c</code>	the weak hardware init hooks
<code>src/rtos/os-main.cpp</code>	the default <code>main()</code> that runs <code>os_main()</code>
<code>src/diag/trace*.cpp</code>	the trace backends (semihosting, RTT, ITM, POSIX)
<code>src/semihosting/c-syscalls-semihosting.cpp</code>	newlib syscalls over semihosting
<code>tests/T0-CHECK.md</code>	recorded intermittent failures and caveats
<code>tests/deprecated/</code>	the old, unbuilt tests and scripts
<code>config/.clang-format</code> , <code>config/.cmake-format.py</code>	style rules
<code>scripts/clang-format.sh</code> , <code>scripts/cmake-format.sh</code>	the formatting actions
<code>.vscode/launch.json</code>	native gdb launch configurations
<code>@micro-os-plus/build-helper/cmake/*</code>	the <code>xpack_*</code> functions and toolchain files

Appendix B — glossary

Term	Meaning
action	A named shell command in <code>package.json</code> , run with <code>xpm run</code> .
build configuration	platform + toolchain + build type; owns a build folder.
CTest	CMake's test runner; <code>add_test()</code> registers, <code>ctest</code> runs.
interface library	A CMake target that only propagates usage requirements.
xPack	The package format used by xpm.
xpm	The Node.js package/build manager for C/C++ projects.
semihosting	A debug protocol that lets the target use the host's I/O.
toolchain file	A CMake file that selects the compilers for cross builds.